

Projeto de Algoritmos – Controle Financeiro Familiar

Integrantes do Grupo

- Daniel Koyama
 - Eduardo Saes
 - Lucas Martins
 - Nicolay Oliveira
 - Yasmin Miranda
-

1. Introdução

O projeto “Controle Financeiro Familiar” foi desenvolvido em Python com o objetivo de auxiliar duas pessoas no controle financeiro doméstico, permitindo o cálculo da renda total familiar, divisão proporcional de gastos e visualização gráfica da participação de cada pessoa na renda.

O sistema possui uma interface gráfica desenvolvida com a biblioteca Tkinter e utiliza a biblioteca Matplotlib para geração de gráficos.

O software foi criado com foco na aplicação prática dos conceitos estudados na disciplina de Algoritmos, utilizando entrada e saída de dados, estruturas de repetição, estruturas de decisão, vetores e interface gráfica.

2. Problema Proposto

Muitas famílias possuem dificuldade em organizar despesas compartilhadas de forma justa, principalmente quando cada pessoa possui uma renda diferente.

O projeto resolve esse problema realizando automaticamente:

- O cálculo da renda total.
- A divisão proporcional dos gastos.
- O armazenamento do histórico de despesas.
- A exibição visual da participação financeira de cada pessoa.

3. Objetivo do Sistema

O principal objetivo do sistema é facilitar o controle financeiro familiar por meio de uma interface simples e intuitiva.

O programa permite:

- Inserir a renda de duas pessoas.
 - Calcular a renda total.
 - Adicionar gastos.
 - Dividir os gastos proporcionalmente.
 - Visualizar histórico de despesas.
 - Exibir gráfico da proporção da renda.
-

4. Tecnologias Utilizadas

Linguagem

- Python

Bibliotecas

- Tkinter → Interface gráfica.
 - Matplotlib → Geração do gráfico de pizza.
-

5. Conceitos de Algoritmos Aplicados

Entrada e saída de dados

O programa recebe dados digitados pelo usuário, como:

- Renda da pessoa 1.
- Renda da pessoa 2.
- Valor do gasto.
- Descrição do gasto.

Os resultados são exibidos na tela por meio de labels e histórico.

Estruturas de decisão

O sistema utiliza estruturas condicionais para validar situações específicas.

Exemplo:

```
if valor > renda_total:
    label_restante.config(
        text= ("Gasto maior que a renda restante!")
```

Essa condição impede que o usuário registre um gasto maior do que a renda disponível.

Estruturas de repetição

Foi utilizado o laço de repetição **for** para percorrer o vetor de gastos e exibir o histórico.

```
for gasto in gastos:
    texto = (f"{gasto[0]}: -R$ {gasto[1]:.2f}\n"
            f"Pessoa 1 paga: R$ {gasto[2]:.2f}\n"
            f"Pessoa 2 paga: R$ {gasto[3]:.2f}\n\n")
```

Vetores

O sistema utiliza vetores para armazenar as informações utilizadas no gráfico, guardando os valores das rendas e os nomes das pessoas.

Exemplo:

```
valores = [renda1, renda2]
nomes = ["Pessoa 1", "Pessoa 2"]
```

Esses vetores são utilizados pela biblioteca Matplotlib para gerar o gráfico de pizza com a participação financeira de cada pessoa.

6. Funcionamento do Sistema

Passo 1 – Inserção das rendas

O usuário informa a renda das duas pessoas.

O sistema calcula automaticamente:

$\text{renda_total} = \text{renda1} + \text{renda2}$

Passo 2 – Exibição do gráfico

Após calcular as rendas, o sistema gera um gráfico de pizza mostrando a proporção da renda de cada pessoa.

Exemplo:

- Pessoa 1 → 70%
 - Pessoa 2 → 30%
-

Passo 3 – Adição de gastos

O usuário informa:

- Valor do gasto.
- Descrição.

O programa divide automaticamente o valor proporcionalmente.

Exemplo:

Se:

- Pessoa 1 ganha R\$ 7000
- Pessoa 2 ganha R\$ 3000
- Gasto = R\$ 1000

Então:

- Pessoa 1 paga R\$ 700
 - Pessoa 2 paga R\$ 300
-

Passo 4 – Histórico

Todos os gastos ficam registrados no histórico do sistema.

O usuário também pode limpar o histórico utilizando o botão “Limpar Histórico”; Além disso, o sistema utiliza uma estrutura de matriz para armazenar os gastos do casal.

Cada linha da matriz guarda:

- Descrição do gasto.
- Valor total.
- Valor pago pela pessoa 1.
- Valor pago pela pessoa 2.

Sempre que um novo gasto é adicionado, o sistema cria uma nova linha na matriz seguindo o mesmo padrão de informações.

Exemplo:

```
gastos.append([descricao, valor, paga1, paga2])
```

Passo 5 – Botão de Reset

Recentemente foi adicionada uma nova funcionalidade ao sistema: o botão de reset.

Esse botão é responsável por:

- Voltar todas as variáveis para 0.
- Limpar todo o histórico de gastos.
- Limpar o gráfico exibido na interface.
- Resetar todas as labels do sistema.
- Retornar para a interface inicial de inserção dos salários.

Essa funcionalidade facilita o reinício completo do controle financeiro sem precisar fechar o programa.

7. Estrutura do Código

O sistema foi dividido em funções para melhorar a organização.

Funções principais

`atualizar_grafico()`

Responsável por criar e atualizar o gráfico de pizza.

calcular_renda()

Realiza:

- Leitura das rendas.
- Soma da renda total.
- Atualização da interface.

subtrair_gasto()

Responsável por:

- Ler os gastos.
- Validar valores.
- Dividir proporcionalmente.
- Atualizar histórico.

limpar_historico()

Apaga todos os gastos registrados.

8. Interface Gráfica

A interface foi construída utilizando Tkinter.

Ela foi organizada em três áreas principais:

Área esquerda

- Inserção de rendas.
- Inserção de gastos.
- Exibição da renda restante.

Área direita

- Histórico de gastos.

Área lateral

- Gráfico de pizza.

9. Pontos Positivos do Projeto

- Interface simples e intuitiva.
 - Fácil utilização.
 - Aplicação prática para o cotidiano.
 - Uso de gráficos para melhor visualização.
 - Organização modular do código.
-

10. Melhorias Futuras

Algumas melhorias que podem ser adicionadas futuramente:

- Salvamento de dados em banco de dados.
 - Relatórios mensais.
 - Exportação em PDF.
 - Gráficos mais avançados.
 - Ler extratos
-

11. Como Executar o Projeto

Instalar dependências

Abra o terminal e execute:

```
pip install matplotlib
```

O Tkinter normalmente já vem instalado com Python.

Executar o programa

Salve o arquivo como:

```
interface2.py
```

Depois execute:

12. Conclusão

O projeto permitiu aplicar na prática os principais conceitos estudados na disciplina de Algoritmos.

Além da parte lógica, o grupo conseguiu desenvolver uma interface gráfica funcional e intuitiva, tornando o sistema mais amigável para o usuário.

O trabalho também contribuiu para o aprendizado de programação em Python, organização de código, trabalho em equipe e desenvolvimento de soluções reais para problemas do cotidiano.